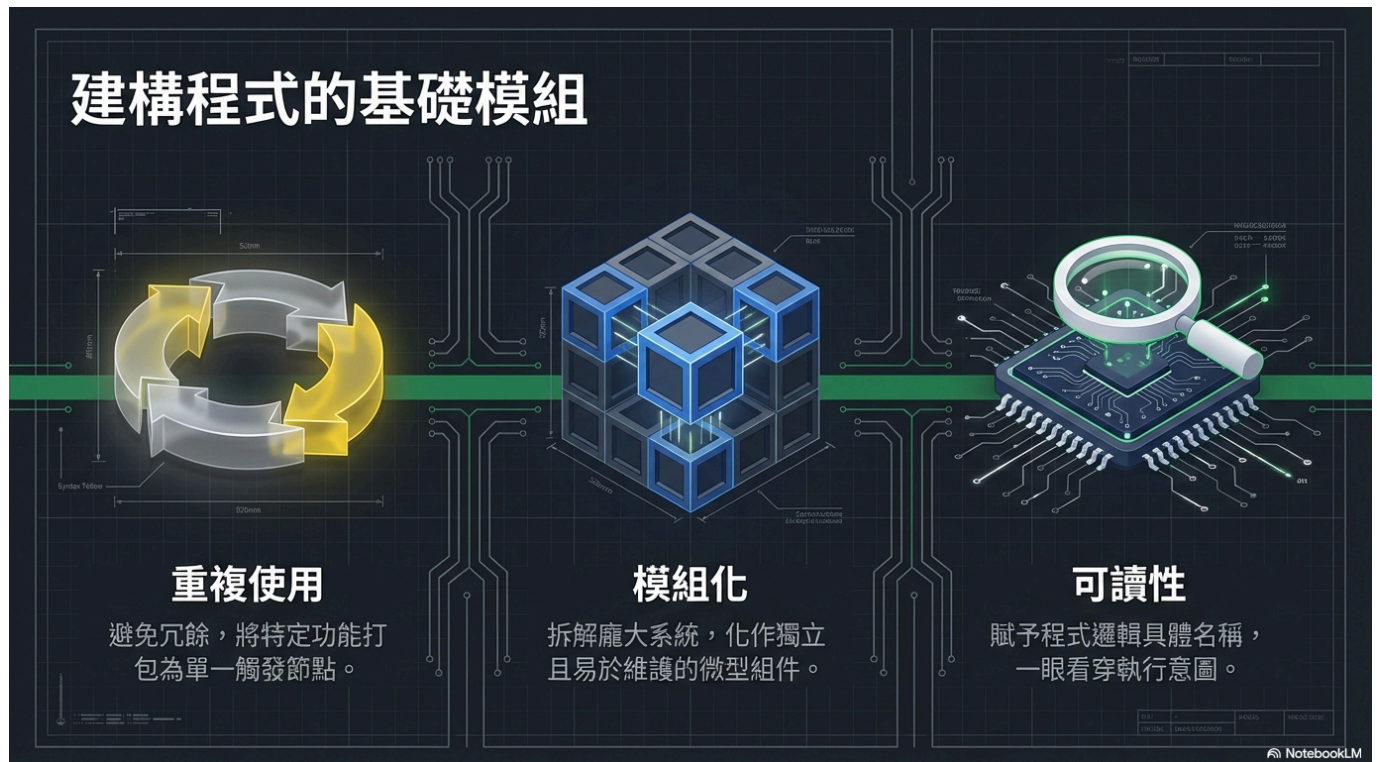


函數是一個可以重複使用的程式碼區塊。主要目的是將一段特定的功能打包起來，當你需要執行這個功能時，只要呼叫這個函數就行了。

使用函數有以下幾個主要優點：



定義&呼叫函數

```
# 定義函數
def my_function():
    # 這是函數內部的程式碼
    print("這是一個簡單的函數。")

# 呼叫函數
my_function()
```

參數 (Parameters)

參數就像是函數的輸入。你可以在呼叫函數時，傳遞資料給它，讓函數可以根據這些資料執行不同的動作。

單一參數

```
def greeting(name):
    print("嗨，", name, "早安！")

greeting("維誠")
```

多個參數

```
def subtract(x1, x2):  
    result = x1 + x2  
    print(result)  
  
subtract(7, 3)
```

Keyword Arguments

```
# 使用關鍵字參數，順序可以不固定  
def interest(interest_type, subject):  
    print("主題是:", subject)  
    print("興趣是:", interest_type)  
  
interest(subject="程式設計", interest_type="Python")
```

預設參數

```
# 如果呼叫時沒給 msg，它就會自動帶入 "早安"  
def greet_with_default(name, msg="早安"):  
    print("嗨，", name, msg, "！")  
  
greet_with_default("維誠") # 情況 1：不提供第二個參數，使用預設值  
greet_with_default("維誠", "晚安") # 情況 2：提供第二個參數，覆蓋預設值
```

函數參數是串列

Python也可以將list作為參數傳遞給函數。讓函數可以處理一組資料，而不是單一的數值。

```
def product_msg(customers):  
    for customer in customers: # 這裡的 customers 是一個串列  
        msg = 'str2' + customer + 'str2'  
        print(msg)  
  
customers = ['喵喵', '旺旺', '呱呱']  
product_msg(customers)
```

補充：傳遞任意數量的參數：*args 與 **kwargs

*args

使用 * 加上一個參數名（通常用 *args），可以讓函數接收任意數量的位置參數。這些參數會被打包成一個元組（tuple）。

```
def make_icecream(*args):  
    print("冰淇淋的配料：")  
    for item in args:  
        print(item)  
  
make_icecream("草莓醬", "OREO餅乾", "巧克力碎片")
```

**kwargs

使用 ** 加上一個參數名（通常用 **kwargs），可以讓函數接收任意數量的關鍵字參數。這些參數會被打包成一個字典（dictionary）。

```
def build_dict(**kwargs):  
    print(kwargs['name'])  
  
build_dict(name='James', Age='32', City='Cleveland', State='Ohio')
```

合併寫法：*args 與 **kwargs

```
def build_dict(*args, **kwargs):  
    print(args)  
    print(kwargs)  
    return kwargs  
  
player_dict = build_dict('James', Age='32', City='Cleveland', State='Ohio')  
print(player_dict)
```

練習：練習題目_Parameters

回傳值

要使用 return 關鍵字，函數可以將計算結果或資料回傳給呼叫它的地方。

無回傳值

```
def greeting(name):  
    print("嗨，" + name + "早安！")  
  
result = greeting("Helena")  
print(result)
```

單一回傳值

```
def greeting(name):  
    return "嗨，" + name + "早安！"  
  
result = greeting("Helena")  
print(result)
```

多個回傳值

Python 的函數可以同時回傳多個值，這些值會被打包成一個元組 (tuple)

```
def multifunc(x1, x2):  
    addresult = x1 + x2  
    subresult = x1 - x2  
    return addresult, subresult  
  
# 呼叫函數並接收回傳的多個值  
add, sub = multifunc(10, 2)  
print("加法結果:", add)  
print("減法結果:", sub)
```

補充：函數可以做為回傳

一個函數也可以回傳另一個函數，這在進階程式設計中很常用。

```
def create_multiplier(number):  
    def multiplier(num):  
        return number * num  
    return multiplier  
  
double_function = create_multiplier(2)  
triple_function = create_multiplier(3)  
  
print("兩倍數字:", double_function(5))  
print("三倍數字:", triple_function(5))
```

練習：練習題目_Return

Docstring

Docstring 是一種良好的程式設計習慣。它位於函數定義的第一行，以三個引號 `"""` 包裹，用來清楚描述函數的用途與功能。

- 自我說明：程式本身就能解釋功能，減少額外註解的需求。
- 協作便利：其他人閱讀程式時，可以快速理解函數的目的。
- 維護效率：在程式更新或擴充時，Docstring 提供清晰的功能定位。

```
def calculate_area(width, height):  
    """  
    計算矩形的面積。  
  
    參數：  
        width (float): 矩形的寬度。  
        height (float): 矩形的高度。  
  
    回傳：  
        float: 矩形的面積。  
    """  
    return width * height
```

補充：函數是物件

函數本身就是一種物件，這意味著你可以將函數賦予給變數，或者作為參數傳遞給另一個函數。

```
def add(x, y):  
    return x + y  
  
def mul(x, y):  
    return x * y  
  
def running(func, arg1, arg2):  
    return func(arg1, arg2)  
  
result = running(add, 5, 10)  
print(result)
```

練習：練習題目

Lambda：匿名函數

顧名思義就是沒有名稱的函數。它們通常只用於一個簡單的、一次性的任務。

```
lambda 參數: 運算式  
# 參數：是輸入，可以有多個，用逗號分隔。  
# 運算式：是函數體，只能有一個運算式，且其結果會被自動回傳，不用 return
```

lambda基本運算

```
# 平方Lambda 函數  
square = lambda x: x ** 2  
print(square(10))
```

```
# 定義一個將兩個數相加的 Lambda 函數
add = lambda x, y: x + y
print(add(5, 3)) # 輸出: 8

# 定義一個將數字乘以 3 的 Lambda 函數
multiply_by_three = lambda x: x * 3
print(multiply_by_three(4)) # 輸出: 12
```

結合多個參數和運算

```
# 計算 (x + y) * z 的 Lambda 函數
complex_calc = lambda x, y, z: (x + y) * z
print(complex_calc(2, 3, 4)) # 輸出: 20
```

lambda條件運算

```
# 返回兩個數中較大的值
max_num = lambda x, y: x if x > y else y
print(max_num(10, 15)) # 輸出: 18
print(max_num(20, 5)) # 輸出: 20
```

lambda其他運算

```
### 範例：列表依照元素長度排序
# 使用 Lambda 作為 sorted() 的 key 函數，按特定條件排序。
words = ["apple", "banana", "cherry"]
sorted_words = sorted(words, key=lambda x: len(x))
print(sorted_words) # 輸出: ['apple', 'cherry', 'banana']
```

```
### 範例：列表忽略大小寫排序
words = ["Banana", "apple", "Cherry", "date"]

sorted_words = sorted(words, key=lambda x: x.lower())
print(sorted_words)
# 輸出: ['apple', 'Banana', 'Cherry', 'date']
```

```
### 範例：列表依數字的絕對值排序
# 範例：依數字的絕對值排序
numbers = [-10, 5, -3, 8, -1]
sorted_numbers = sorted(numbers, key=lambda x: abs(x))
```

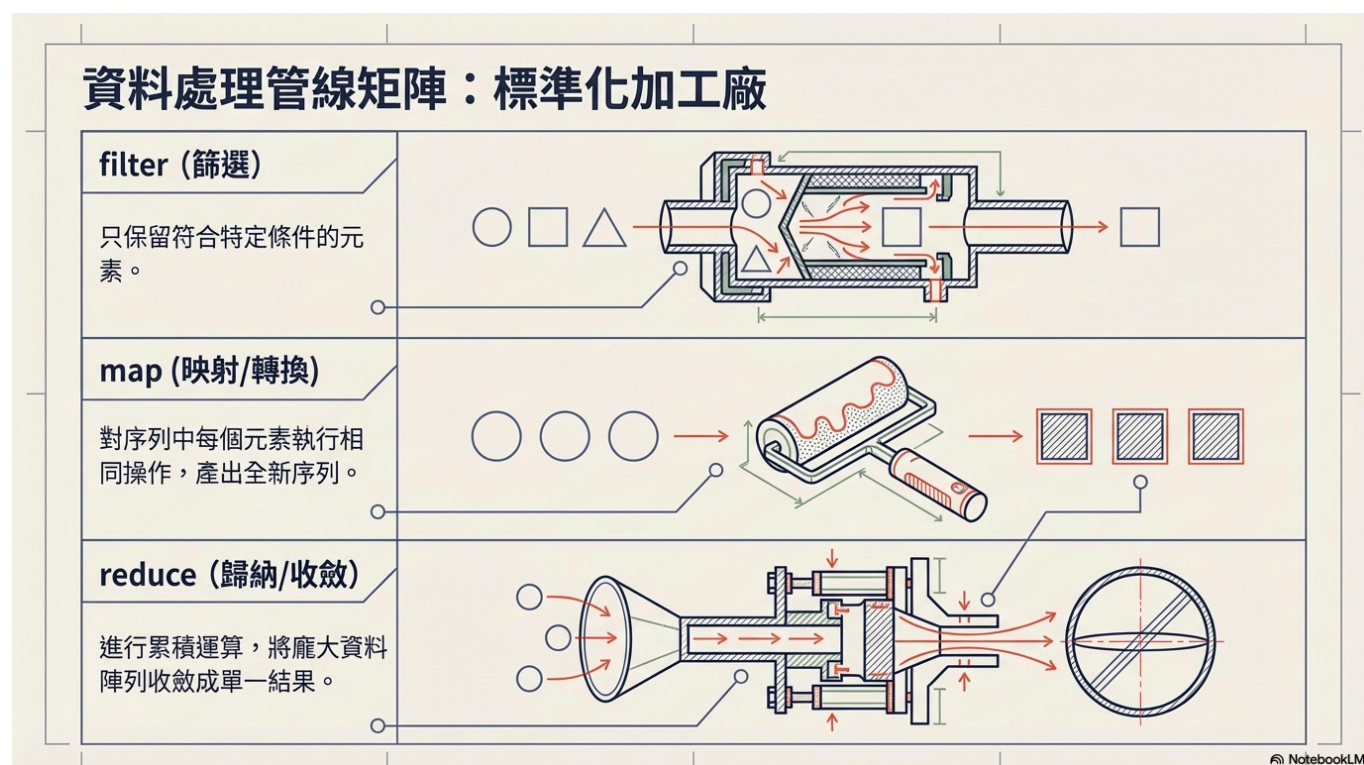
```
print(sorted_numbers)
# 輸出: [-1, -3, 5, 8, -10]
```

補充：高階函數

高階函數指的是能夠 接收其他函數作為參數，或 將函數作為回傳值 的函數。這種設計讓程式更具彈性與抽象性，能以更簡潔的方式表達複雜的邏輯。

因此經常搭配lambda

- `filter()`：用來篩選符合條件的元素。
- `map()`：將函數套用到每個元素，產生新序列。
- `reduce()`：累積計算序列中的元素，常用於加總或合併。



`filter()`：過濾串列中的元素，只保留符合特定條件的

```
filter(函數, iterable)
# 函數：一個判斷條件的函數，回傳 True 或 False。
# iterable：可迭代的物件，如串列、元組等。
```

```
# 使用 filter() 篩選健康數據
# filter() 用於篩選出符合特定條件的健康數據，例如過濾出心率超過正常範圍（假設正常範圍為 60-100 bpm）的數據。

# 模擬高齡者心率數據列表（單位：beats per minute）
heart_rates = [55, 72, 85, 110, 65, 120, 90]

# 使用 filter 篩選出異常心率 (> 100 或 < 60)
```

```
abnormal_rates = list(filter(lambda x: x > 100 or x < 60, heart_rates))
print(abnormal_rates) # 輸出: [55, 110, 120]
```

map(): 對串列中的每個元素執行相同的操作，並回傳新的結果

```
map(函數, iterable)
# 函數：一個處理每個元素的函數。
```

```
# 使用 map() 轉換健康指標
# map() 用於對每個健康數據執行相同的操作，例如將攝氏溫度轉換為華氏溫度，模擬環境感測器數據處理。

# 模擬高齡者居住環境的攝氏溫度列表
celsius_temps = [22, 25, 18, 30]

# 使用 map 將攝氏轉換為華氏 (°F = °C * 9/5 + 32)
fahrenheit_temps = list(map(lambda x: x * 9/5 + 32, celsius_temps))
print(fahrenheit_temps) # 輸出: [71.6, 77.0, 64.4, 86.0]
```

reduce(): 對串列中的元素進行累積運算，將所有元素歸納成一個單一值。

```
from functools import reduce

reduce(函數, iterable)
# 函數：必須有兩個參數，分別是累積值和下一個元素。
```

```
from functools import reduce

# 模擬一天內多次測量的血氧濃度數據
blood_oxygen = [95, 97, 94, 96, 98]

# 使用 reduce 計算總和
total_oxygen = reduce(lambda x, y: x + y, blood_oxygen)
average_oxygen = total_oxygen / len(blood_oxygen)
print(f"平均血氧濃度: {average_oxygen}") # 輸出: 平均血氧濃度: 96.0
```

變數作用域 (Scope)

Scope 決定了該變數在程式中可被存取的範圍。理解作用域能避免命名衝突，並提升程式的可讀性與維護性。

- 區域變數 (Local Variables)：在函數內部定義，只能在該函數內使用。當函數執行結束後，區域變數也會消失。

- 全域變數 (Global Variables)：在所有函數之外定義，可在程式的任何地方存取。若要在函數內修改全域變數，必須使用 `global` 關鍵字，明確告訴程式你要操作的是全域範圍的變數。

```
global_msg = "Global Variable"

def printMsg():
    # 可以在函數內部讀取全域變數
    print("函數內可以讀取:", global_msg)

print("主程式可以讀取:", global_msg)
printMsg()
```

pass

在 Python 中，`pass` 是一個空語句，什麼也不做。只作為佔位符，常用於：

- 尚未完成的程式結構：先定義函數或類別框架，避免因為內容空白而報錯。
- 保持語法完整：在需要程式碼但暫時不打算撰寫的地方使用。

```
def calculate_total_price(items):
    pass # 未來會實現計算總價的邏輯，但現在只是佔位
```

```
# 空的條件語句
age = 20
if age >= 18:
    pass # 未來會加入成年後的處理邏輯
else:
    print("未成年")
```

```
# 空的迴圈
for i in range(5):
    pass # 未來會加入迴圈內的處理邏輯
```

```
# 空的類定義
class MyClass:
    pass # 未來會加入類的屬性和方法
```

專案

- [基於gradio實作猜數字遊戲](#)
- [基於gradio實作blackjack](#)

- 基於gradio實作點餐系統
- 基於gradio實作規則版聊天機器人
- 基於gradio實作簡易ATM系統

Zerojudee練習題目

